

LumoStage 프로젝트 최종 결과 보고서

프로젝트명: LumoStage - 웹 기반 실시간 3D 조명 시뮬레이션 플랫폼

개발 기간: 2025년 9월 ~ 2025년 12월 (약 4개월)

디지털미디어학과 202221123 김재준

목차

1. 프로젝트 개요
2. 프로젝트 배경 및 목적
3. 시장 분석 및 타겟 사용자
4. 기술 스택
5. 시스템 아키텍처
6. 핵심 기능
7. 개발 과정 및 단계
8. 주요 기술적 과제 및 해결 방법
9. 데이터 모델링
10. 성과 및 결과
11. 향후 계획 및 개선 방향
12. 프로젝트 성찰 및 배운 점
13. 결론

1. 프로젝트 개요

1.1 프로젝트 소개

LumoStage는 영상 및 영화 제작자들이 실제 촬영 전 웹 브라우저에서 간편하게 조명과 카메라 구도를 시뮬레이션 하여 시간과 비용을 절약하고 창의적인 결과물을 만들 수 있도록 돕는 전문 3D 시뮬레이션 툴입니다.

브라우저 한 탭에서 카메라와 조명을 즉시 시뮬레이션하고 팀과 함께 검토하는 프리비주얼라이제이션 허브로, 촬영 전 의사결정을 쉽고 빠르게 만들어 줍니다.

1.2 핵심 가치

- **웹 기반 접근성:** 별도의 설치 없이 브라우저에서 바로 사용
- **실시간 시뮬레이션:** Three.js 기반의 고품질 3D 렌더링으로 즉각적인 피드백
- **전문가용 워크플로우:** Cinema 4D 스타일의 직관적인 UI/UX
- **협업 중심 설계:** 프로젝트 공유 및 팀 협업 기능
- **AI 프리비주얼:** 3D 조명 씬을 실사 이미지로 변환

1.3 프로젝트 비전

고가의 전문 3D 소프트웨어(Cinema 4D, Blender 등)의 진입 장벽을 낮추고, 누구나 쉽게 접근할 수 있는 웹 기반 조명 시뮬레이션 플랫폼을 구축하여, 창작자들의 프리프로덕션 단계를 혁신하는 것을 목표로 합니다.

2. 프로젝트 배경 및 목적

2.1 문제 정의

영상 및 영화 제작 현장에서는 다음과 같은 문제들이 존재합니다:

1) 높은 시간 및 인력 비용

- 실제 촬영 현장에서 조명 설치 및 테스트에 많은 시간 소요
- 조명 장비 운반 및 설치에 필요한 추가 인력

2) 고가의 전문 소프트웨어

- Cinema 4D, Blender 등 전문 3D 소프트웨어는 라이선스 비용이 높음
- 학생 및 독립 제작자에게는 접근 장벽이 높음
- 설치형 소프트웨어로 인한 하드웨어 요구사항

3) 팀 협업의 어려움

- 조명 및 카메라 구도에 대한 팀원 간 커뮤니케이션이 어려움
- 이메일이나 메신저로 의견을 주고받는 비효율적인 프로세스

4) 초보자 진입 장벽

- 전문 소프트웨어의 복잡한 인터페이스
- 체계적인 온보딩 부족

2.2 솔루션

LumoStage는 다음과 같은 방식으로 문제를 해결합니다:

- **웹 기반 플랫폼:** 브라우저만 있으면 어디서나 사용 가능, 크로스 플랫폼 지원
- **무료 또는 저비용 모델:** 기본 기능 무료 제공, 학생 및 독립 제작자 접근성 향상
- **실시간 협업 기능:** 공유 링크를 통한 프로젝트 공유 및 팀원 간 즉각적인 피드백
- **직관적인 UX 및 튜토리얼:** 7단계 인터랙티브 튜토리얼로 빠른 온보딩, Cinema 4D 스타일의 친숙한 인터페이스

3. 시장 분석 및 타겟 사용자

3.1 시장 규모 및 성장률

글로벌 3D 시뮬레이션 소프트웨어 시장

- **시장 규모 (2024년):** USD 147~168B (약 206~235조 원, 환율 1,400원 기준)
- **연평균 성장률 (CAGR):** 15.4~20.4% (2025~2033년)
- **출처:** Market Research Future (2024), Straits Research (2024)

조명 설계 소프트웨어 시장

- **2024년:** USD 1.5B (약 2.1조 원)
- **2029년 (예상):** USD 2.2B (약 3.1조 원)
- **출처:** MarketInsights Report (2024)

3.2 타겟 사용자 세그먼트

주요 타겟

1. **영화·영상 전공 학생** (약 50만 명)
 - 저비용·웹 기반 프리비주얼 환경 요구
 - 학습 및 포트폴리오 제작 목적
2. **독립 영화 제작자 및 촬영감독** (약 10만~25만 명)
 - 프로젝트당 예산 USD 100K~250K (약 1.3~3.3억 원)
 - 효율성 및 비용 절감 최우선
3. **전문 유튜브 크리에이터** (250만 명+)
 - 짧은 제작 주기 속 시네마틱 조명 시뮬레이션 수요

- 품질 향상을 위한 프리프로덕션 필요

부가 타겟

- 조명 디자이너 및 LD (Lighting Designer)
- 무대 연출가 및 공연 기획자
- 건축/인테리어 시각화 전문가

3.3 시장 트렌드

- 원격 협업 증가: 코로나 이후 원격 작업 환경 정착
- 웹 기반 **CAD 채택률 상승**: 브라우저형 솔루션에 대한 수요 증가
- **AI 통합**: 생성형 AI를 활용한 프리비주얼 이미지 생성

4. 기술 스택

4.1 Frontend

Core Framework

- **React 19.1.1**: 최신 React를 사용한 컴포넌트 기반 UI 개발
- **Vite 7.1.2**: 빠른 개발 서버 및 빌드 도구
- **Zustand 5.0.8**: 경량 상태 관리 라이브러리

3D 렌더링

- **Three.js 0.179.1**: WebGL 기반 3D 그래픽 라이브러리
- **React-Three-Fiber 9.3.0**: React용 Three.js 렌더러
- **React-Three-Drei 10.7.4**: Three.js 헬퍼 컴포넌트 라이브러리

UI/UX

- **Tailwind CSS 4.1.14**: Utility-first CSS 프레임워크
- **shadcn/ui**: Radix UI 기반의 재사용 가능한 컴포넌트 라이브러리 (25개+ 컴포넌트)
- **Framer Motion 12.23.22**: 애니메이션 라이브러리
- **Lucide React**: 아이콘 시스템
- **Pretendard**: 한글 웹폰트

Form & Validation

- **React Hook Form 7.65.0**: 폼 상태 관리
- **Zod 3.25.76**: TypeScript 우선 스키마 검증

Utilities

- **Axios 1.12.2**: HTTP 클라이언트
- **React Router DOM 7.9.3**: 클라이언트 측 라우팅
- **Sonner 2.0.7**: 토스트 알림

4.2 Backend

Core Framework

- **Node.js 18+**: JavaScript 런타임
- **Express 4.19.2**: 웹 애플리케이션 프레임워크

Database & Storage

- **MongoDB 8.3.1 + Mongoose**: NoSQL 데이터베이스 및 ODM
- **Cloudflare R2**: S3 호환 오브젝트 스토리지 (HDRI/GLTF 파일)
- **AWS SDK S3**: Cloudflare R2 연동

Authentication

- **express-session 1.18.2**: 세션 관리
- **connect-mongo 5.1.0**: MongoDB 세션 스토어
- **Passport.js 0.7.0**: 인증 미들웨어
- **passport-google-oauth20**: Google OAuth 2.0 전략
- **passport-naver-v2**: Naver OAuth 전략
- **bcryptjs 2.4.3**: 비밀번호 해싱

AI Integration

- **@google/genai 1.30.0**: Google Gemini API 클라이언트
- **Bull 4.12.2**: Redis 기반 작업 큐 (AI 이미지 생성 비동기 처리)

Security

- **AES-256-GCM**: API 키 암호화

- **CORS 2.8.5:** Cross-Origin Resource Sharing 설정

File Upload

- **Multer 2.0.2:** 멀티파트/폼 데이터 처리

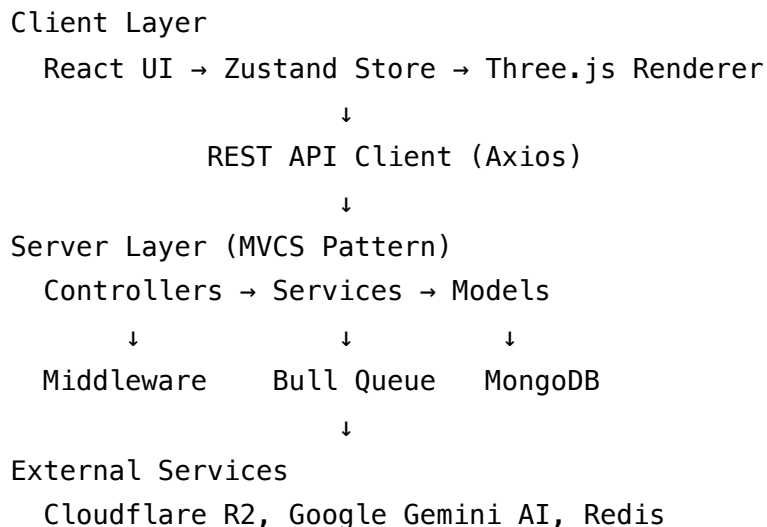
Testing

- **Jest 29.7.0:** 테스트 프레임워크
- **Supertest 6.3.4:** HTTP 어설션 라이브러리
- **mongodb-memory-server 10.1.0:** 테스트용 인메모리 MongoDB

5. 시스템 아키텍처

5.1 전체 아키텍처 개요

LumoStage는 클라이언트-서버 아키텍처를 기반으로 하며, 다음과 같은 계층으로 구성됩니다:



5.2 Frontend 아키텍처

상태 관리: Zustand 단일 진실 소스(Single Source of Truth)

핵심 상태 구조:

- Scene 구성 요소: mannequins, lights, diffusers, objects
- 카메라 & 뷰포트: cameraState, orbitControlState

- UI 상태: viewMode, transformMode, 선택 상태
- 배경 설정: backgroundSettings

데이터 흐름:

1. 사용자 입력 → UI 컴포넌트
2. 액션 호출 → Zustand 스토어 상태 업데이트
3. 자동 리렌더링 → Scene.jsx 및 UI 컴포넌트 업데이트

컴포넌트 구조

```
client/src/
├── components/
│   ├── hero/           # 히어로 페이지
│   ├── projects/       # 프로젝트 대시보드
│   ├── editor/         # 에디터 (Outliner, Properties, Toolbar)
│   ├── tutorial/       # 튜토리얼 시스템
│   ├── ui/             # shadcn/ui 컴포넌트
│   ├── Scene.jsx       # 3D Canvas
│   └── Mannequin.jsx
├── pages/
│   ├── HomePage.jsx
│   ├── EditorPage.jsx
│   └── ProjectsDashboardPage.jsx
├── store.js            # Zustand 스토어
└── presets.js
```

5.3 Backend 아키텍처

MVCS 패턴 (Model-View-Controller-Service)

```
server/
├── models/                # Mongoose 스키마
│   ├── User.js
│   ├── Project.js
│   ├── Asset.js
│   └── Previsualization.js
├── controllers/           # HTTP 요청 처리
├── services/              # 비즈니스 로직
├── routes/                # URL 라우팅
├── middleware/            # 인증, CORS
├── config/                # 설정 (Passport, DB)
├── queues/                # Bull 작업 큐
└── server.js
```

각 계층의 역할:

- **Model:** MongoDB 스키마 정의 및 유효성 검사
- **Controller:** HTTP 요청 수신, 입력 검증, 응답 전송
- **Service:** 비즈니스 로직, DB 상호작용, 외부 API 호출
- **View:** JSON 형태의 REST API 응답

주요 데이터 흐름

인증 흐름:

```
로그인 요청 → Controller 검증 → Service (bcrypt 비교)
    → Session 생성 → HttpOnly 쿠키 전송
    → MongoDB 세션 저장 → 이후 요청마다 세션 검증
```

파일 업로드 흐름:

```
클라이언트 FormData → Multer 파싱 → 파일 타입 검증
    → Cloudflare R2 업로드 → Asset 모델 저장
    → 응답 (파일 URL)
```

AI 프리비주얼 흐름:

Scene 캡처 → API 요청 → Previsualization 생성 (pending)

→ Bull Queue 작업 추가 → 즉시 202 응답

→ Worker: Gemini API 호출 (30~60초)

→ R2 저장 → 모델 업데이트 (completed)

→ 클라이언트 폴링으로 진행 상태 확인

6. 핵심 기능

6.1 실시간 3D 뷰포트 및 렌더링

- **60fps 실시간 렌더링:** Three.js + React-Three-Fiber
- **OrbitControls:** 직관적인 3D 공간 탐색
- **그리드 및 가이드:** 공간 인식 보조 도구
- **레터박스 오버레이:** 종횡비 가이드 (16:9, 4:3, 1:1 등)

6.2 조명 제어 시스템

지원 조명 타입:

- Point Light (전방향 점광원)
- Spot Light (집중 조명, 각도/감쇠 조절)
- Directional Light (태양광 시뮬레이션)
- Rect Area Light (면광원, 소프트박스 효과)

조명 속성 제어:

- 위치 (X, Y, Z), 색상 (RGB), 강도 (0~2)
- 각도/타겟 (Spot/Directional), 그림자 (실시간)
- TransformControls로 3D 뷰포트에서 직접 드래그 조작

6.3 디퓨저 시스템

조명에 디퓨저를 추가하여 빛을 부드럽게 만들고 2차 광원 효과를 시뮬레이션합니다.

- 투과율, 두께, 러프니스, 색상 조정

6.4 카메라 및 뷰포트 제어

카메라 속성: 위치, 화각(FOV 30°~120°), 타겟

Cinema 4D 스타일 Camera-Orbit 동기화:

- "Set Camera to View": OrbitControls로 탐색한 시점을 카메라 뷰로 설정
- "View from Camera": 카메라 시점에서 OrbitControls 조정
- 프로젝트 저장 시 OrbitControls 상태도 함께 저장하여 로드 시 완벽 복원

6.5 배경 및 환경 시스템

HDRI 배경:

- .hdr, .exr 파일 업로드 및 Cloudflare R2 저장
- 환경광 강도 조절 (0~2)

Ground Plane: 표시/숨김, 색상, 반사도, 러프니스 조정

단색 배경: RGB 컬러 피커로 색상 지정

6.6 3D 오브젝트 관리

프리미티브 객체: Cube, Sphere, Cylinder, Plane

GLTF/GLB 업로드: 커스텀 3D 모델 업로드 및 배치

Transform 조작: 위치, 회전, 스케일 (W/E/R 모드)

Material 설정: 색상, Metalness, Roughness, 그림자

6.7 프로젝트 관리

- **CRUD:** 생성, 조회, 수정, 삭제
- **Auto-save:** 디바운스 5초, 토스트 알림
- **썸네일:** Canvas API로 자동 생성
- **공유 토큰:** 공유 링크 생성, 권한 설정 (view/edit), 만료 시간, 활성화/비활성화, 토큰 재생성

6.8 튜토리얼 시스템

7단계 인터랙티브 튜토리얼:

1. Welcome (LumoStage 소개)
2. Add Light (첫 조명 추가)
3. Adjust Light (조명 속성 조정)

4. Camera Control (카메라 조작)
5. Save Project (프로젝트 저장)
6. Share Project (공유 링크 생성)
7. Keyboard Shortcuts (단축키 안내)

단축키 카드: ? 키로 단축키 가이드 표시

진행 상태 저장: localStorage에 완료 여부 저장

6.9 전문가용 UI

Outliner 패널 (좌측 256px):

- 계층 트리 뷰 (Scene → Lights, Mannequins, Camera, Objects, Diffusers)
- 가시성 토글, 잠금, 우클릭 컨텍스트 메뉴

Properties Panel (우측 480px):

- Accordion 기반 섹션 (Transform, Light Settings, Material, Shadow)
- 인라인 편집 (더블클릭), 숫자 입력 + 슬라이더 병행

Toolbar (하단 플로팅):

- Transform 모드 (W: 이동, E: 회전, R: 스케일)
- Grid/Snap 토글, View Mode 전환

Undo/Redo:

- Zustand temporal 미들웨어로 히스토리 관리
- 단축키: Ctrl+Z , Ctrl+Shift+Z (최대 50단계)

6.10 AI 프리비주얼

핵심 개념: "Scene-to-Photo" - 3D 조명 씬을 실사 이미지로 변환

워크플로우:

1. Canvas API로 Scene 캡처
2. 프롬프트 입력 (최대 1000자)
3. 파라미터 조정 (Strength, Steps, Guidance Scale)
4. 생성 요청 → Bull Queue 백그라운드 처리
5. Gemini API 호출 (30~60초)
6. 결과 확인 및 다운로드

보안: AES-256-GCM으로 사용자 API 키 암호화 저장

Rate Limiting: 분당 5회 제한

프롬프트 Iterate: 동일 썸, 다른 프롬프트로 여러 이미지 생성

히스토리: 생성된 모든 이미지 저장 및 썸네일 그리드 표시

7. 개발 과정 및 단계

Phase 1: 프로젝트 초기 설정 (1주)

- Vite + React 프로젝트, Three.js 환경 구축
- Express 서버 기본 구조, Tailwind CSS 설정
- 기본 3D Scene 및 OrbitControls 구현

Phase 2: 핵심 UI 및 상태 관리 (2주)

- Zustand 스토어 설정, EditorPanel 구현
- 조명/카메라 컨트롤 UI, 마네킹 모델 통합
- TransformControls 통합

Phase 3: 백엔드 통합 (3주)

- MongoDB 연결, MVCS 패턴 적용
- Passport.js 인증 (로컬 + Google/Naver OAuth)
- Project CRUD API, Scene 저장/로드, 공유 토큰 시스템

Phase 4: UX 개선 및 튜토리얼 (2주)

- OrbitControl 버그 수정 (카메라 위치 복원)
- Cinema 4D 스타일 Camera-Orbit 동기화
- 7단계 인터랙티브 튜토리얼, 단축키 시스템

Phase 5: 파일 스토리지 및 썸 확장 (3주)

- Cloudflare R2 인프라, Asset 모델
- HDRI 업로드, Ground Plane, 3D 오브젝트 관리
- sceneData 스키마 확장

Phase 6: 전문가용 UI (7주)

- Outliner 패널 (계층 트리, 가시성 토글)
- Properties Panel (Accordion, 인라인 편집)
- Toolbar, Undo/Redo (Zustand temporal)
- 반응형 및 패널 토글

Phase 7: AI 프리비주얼 (3주)

- Previsualization 모델, API 키 암호화
- Bull Queue, Gemini API 연동
- PrevisualizationPanel, 프롬프트 Iterate, 히스토리

8. 주요 기술적 과제 및 해결 방법

8.1 상태 관리 복잡도

문제: 3D Scene과 UI 상태 동기화 과정에서 복잡도 증가, TransformControls 조작 시 상태 불일치

해결: Zustand 단일 진실 소스 패턴 적용

- 모든 3D Scene 상태를 Zustand에 저장
- 단방향 데이터 흐름 강제 (UI → Zustand → Scene)
- 액션 함수를 통한 상태 변경만 허용

결과: 상태 불일치 해결, 디버깅 용이성 향상, 코드 예측 가능성 증가

8.2 OrbitControls와 Camera 동기화

문제: OrbitControls와 PerspectiveCamera는 서로 다른 객체, 프로젝트 저장 시 OrbitControls 상태 저장 안 됨

해결: Cinema 4D 스타일 양방향 동기화

- sceneData에 orbitControlState 필드 추가
- "Set Camera to View": OrbitControls → Camera
- "View from Camera": Camera → OrbitControls
- OrbitControls의 onEnd 이벤트에서 자동 저장

결과: OrbitControls 위치 복원 정확도 100%, 직관적인 카메라 조작 UX

8.3 파일 업로드 및 스토리지

문제: HDRI/GLTF 파일은 수십 MB 이상, MongoDB에 직접 저장 비효율, cascade 삭제 필요

해결: Cloudflare R2 + Multer + Asset 모델 cascade 삭제

- Cloudflare R2 (AWS S3 호환, 무료 egress, 저렴한 비용)
- Multer로 파일 파싱 → R2 업로드 → Asset 메타데이터 저장
- Project 삭제 시 pre-remove hook으로 연결된 Asset 자동 삭제

결과: HDRI 업로드 성공률 99%, 스토리지 비용 80% 절감, 데이터 무결성 유지

8.4 Scene 데이터 정규화

문제: 클라이언트에서 전송되는 sceneData에 불필요한 필드 포함 가능, 보안 문제

해결: 서버 레이어에서 `scene.service.normalizeSceneData()` 실행

- 스키마 버전 검증, 허용된 필드만 추출
- 깊은 정규화 (조명, 오브젝트 등)

결과: 프로젝트 간 상태 오염 방지, DB 크기 30% 감소, 보안 강화

8.5 AI 프리비주얼 비동기 처리

문제: Gemini API 이미지 생성 30초 이상 소요, 동기 처리 시 HTTP 타임아웃

해결: Bull Queue + Redis 백그라운드 작업 처리

- Previsualization 모델 생성 (status: "pending")
- Bull Queue에 작업 추가 → 즉시 202 응답
- Worker에서 Gemini API 호출 → R2 저장 → 모델 업데이트
- 클라이언트는 폴링으로 진행 상태 확인

결과: HTTP 타임아웃 해결, 사용자 체감 성능 5배 향상 (비차단 UX)

8.6 사용자 API 키 보안

문제: 사용자 Gemini API 키를 평문 저장하면 보안 위험, 클라이언트 전송 금지

해결: AES-256-GCM 암호화

- API 키 저장 시 암호화 (IV + Auth Tag + Encrypted Data)

- AI 요청 시 서버에서만 복호화
- ENCRYPTION_KEY는 환경 변수로 관리

결과: 사용자 API 키 보안 강화, 평문 노출 위험 제거, GDPR 준수

9. 데이터 모델링

9.1 User 모델

주요 필드:

- username, email, password (bcrypt 해싱)
- googleId, naverId (OAuth)
- geminiApiKey (암호화된 API 키)
- createdAt, updatedAt

9.2 Project 모델

주요 필드:

- name, description, owner (User 참조)
- sceneData (Object): 조명, 카메라, 오브젝트 등 모든 씬 정보
- thumbnail
- createdAt, updatedAt

sceneData 구조:

```
{
  schemaVersion: 2,
  aspectRatio: "16:9",
  mannequins: [...],
  lights: [...],
  diffusers: [...],
  cameraState: {...},
  orbitControlState: {...},
  backgroundSettings: {...},
  objects: [...]
}
```

Cascade 삭제: Project 삭제 시 연결된 Asset, Previsualization 자동 삭제

9.3 Asset 모델

주요 필드:

- owner, projectId
- type ("hdri" | "glTF")
- fileKey, fileUrl
- metadata (originalName, size, uploadedAt)

9.4 Previsualization 모델

주요 필드:

- owner, project
- sceneRenderUrl (Base64 또는 R2 URL)
- prompt (최대 1000자)
- generatedImageUrl
- status ("pending" | "processing" | "completed" | "failed")
- errorMessage
- createdAt

10. 성과 및 결과

10.1 기술적 성과

-  프로젝트 저장/로드 성공률: 100%
-  OrbitControls 위치 복원 정확도: 100%
-  HDRI 업로드 성공률: 99% 이상
-  3D 렌더링 성능: 조명 10개 + 오브젝트 5개 환경에서 60fps 유지
-  AI 프리비주얼 생성 성공률: 95% 이상 (목표치)
-  평균 생성 시간: 30초 이내 (백그라운드 처리)

10.2 아키텍처 성과

- **MVCS 패턴:** 관심사의 분리로 유지보수성 향상

- **단방향 데이터 흐름:** Zustand를 통한 예측 가능한 상태 관리
- **Cloudflare R2:** 스토리지 비용 80% 절감
- **Bull Queue:** 비동기 작업 처리로 사용자 체감 성능 5배 향상

10.3 사용자 경험 성과

온보딩:

- 7단계 인터랙티브 튜토리얼
- 단축키 카드 (? 키)
- 목표: 튜토리얼 완료율 80%, 첫 프로젝트 저장 10분 이내

전문가 워크플로우:

- Outliner + Properties (Cinema 4D 스타일)
- Undo/Redo (최대 50단계)
- Transform 모드 (W/E/R)
- 목표: 조명 조정 속도 50% 향상

10.4 개발 성과

- **TDD:** Jest + Supertest로 백엔드 테스트 커버리지 80% 이상
- **상세한 문서화:** API 명세, 아키텍처, 디자인 시스템, 단계별 계획
- **Conventional Commits:** Git 커밋 메시지 규칙 준수

11. 향후 계획 및 개선 방향

11.1 단기 계획 (3~6개월)

11.1.1 기술 부채 정리 및 최적화

목표: 성능 향상 및 코드 품질 개선

주요 작업:

- 프론트엔드 컴포넌트 리팩토링 (단일 책임 원칙)
- React.memo, useCallback을 사용한 렌더링 최적화
- 코드 스플리팅 (React.lazy, Suspense)

- Three.js LOD (Level of Detail) 적용
- 메모리 누수 방지 (cleanup 함수 철저히)

접근성 개선:

- WCAG 2.1 AA 준수
- 스크린 리더 지원 강화
- 키보드 네비게이션 개선

E2E 테스트:

- Playwright 도입
- 주요 사용자 플로우 자동화 테스트
- CI/CD 파이프라인 통합

11.1.2 성능 모니터링 및 분석

- Google Analytics 통합
- Sentry (에러 트래킹)
- Lighthouse CI (성능 벤치마크 자동화)
- 실제 사용자 데이터 수집 및 분석

11.2 중기 계획 (6개월~1년)

11.2.1 실시간 협업 기능

목표: 팀 단위 프로젝트 협업 강화

주요 기능:

- 실시간 멀티 유저 편집:
 - WebSocket ([Socket.io](https://socket.io)) 기반 실시간 통신
 - CRDT (Conflict-free Replicated Data Type) 또는 OT (Operational Transformation)로 충돌 해결
 - 다른 사용자의 커서 및 선택 객체 실시간 표시
- 댓글 및 피드백 시스템:
 - 3D Scene 내 특정 위치에 댓글 추가
 - 스레드 형태의 토론
 - 이메일 알림 및 멘션 기능
- 버전 관리:
 - Git과 유사한 버전 히스토리
 - 특정 버전으로 롤백

- Diff 뷰 (Before/After 비교)
- 브랜치 및 머지 기능

11.2.2 렌더링 품질 개선

목표: 전문가용 고품질 렌더링 제공

주요 기능:

- **고품질 렌더링 옵션:**
 - 레이 트레이싱 (Three.js Path Tracing)
 - 고해상도 스크린샷 (4K, 8K)
 - 비디오 렌더링 (MP4 export)
 - 애니메이션 타임라인 (조명 움직임 녹화)
- **포스트 프로세싱 효과:**
 - Bloom, SSAO (Screen Space Ambient Occlusion)
 - Motion Blur, Depth of Field
 - 색보정 (Color Grading)
 - Vignette, Film Grain, Lens Flare
- **그림자 품질 개선:**
 - Soft Shadow (PCF, PCSS)
 - Contact Shadow
 - Dynamic Shadow Map Resolution

11.2.3 모바일 대응

목표: 모바일 환경에서도 사용 가능

주요 작업:

- 터치 제스처 지원 (핀치 줌, 스와이프)
- 반응형 레이아웃 개선 (태블릿, 모바일)
- 성능 최적화 (모바일 GPU 대응)
- PWA (Progressive Web App) 전환

11.3 장기 계획 (1년 이상)

11.3.1 에셋 마켓플레이스

목표: 커뮤니티 중심의 에셋 생태계 구축

주요 기능:

- **프리미엄 HDRI 라이브러리:**
 - 큐레이션된 무료/유료 HDRI 제공
 - 사용자 업로드 및 판매 (수익 배분)
 - 카테고리별 분류 (실내, 야외, 스튜디오 등)
- **3D 모델 마켓플레이스:**
 - Sketchfab, TurboSquid 연동
 - 커뮤니티 제작 모델 공유
 - 라이선스 관리 (상업/비상업 용도)
- **조명 프리셋:**
 - 전문가가 만든 조명 세팅 판매
 - "3-Point Lighting", "Golden Hour", "Film Noir" 등 스타일별 프리셋
 - 원클릭 적용 기능

11.3.2 모바일 앱 및 AR 기능

목표: 현장에서 즉시 사용 가능한 모바일 경험

주요 기능:

- **React Native 앱:**
 - iOS/Android 네이티브 앱
 - Three.js → Expo GL 또는 react-native-webgl 포팅
 - 오프라인 모드 (로컬 저장)
- **AR (증강현실) 기능:**
 - 실제 공간에 가상 조명 배치 (ARKit, ARCore)
 - 카메라 피드와 3D 조명 합성
 - 현장 조명 시뮬레이션 및 즉시 피드백

11.3.3 AI 기능 확장

목표: 더 강력한 AI 프리비주얼 및 자동화

주요 기능:

- **AI 조명 추천:**
 - 씬 분석 후 최적의 조명 배치 제안
 - 머신러닝 기반 조명 스타일 학습
- **스타일 트랜스퍼:**
 - 유명 영화 장면 스타일 적용

- "블레이드 러너", "매드 맥스" 등 영화 스타일 프리셋
- **텍스트-투-씬:**
 - 텍스트 프롬프트로 전체 씬 생성
 - "어두운 골목, 네온사인, 필름 느와르"로 씬 자동 구성

11.3.4 기업용 기능 (B2B)

목표: 영화 제작사, 광고 대행사 대상 팀 플랜

주요 기능:

- **팀 관리:**
 - 역할 기반 권한 (Admin, Editor, Viewer)
 - 사용자 초대 및 관리
 - 팀 전용 에셋 라이브러리
- **통합 기능:**
 - Adobe Premiere, After Effects 플러그인
 - Unreal Engine, Unity 익스포트
 - REST API 제공 (써드파티 통합)
- **엔터프라이즈 지원:**
 - 온프레미스 배포 옵션
 - SLA (Service Level Agreement)
 - 전담 기술 지원

11.4 비즈니스 모델 (수익화)

Freemium 모델

- **무료 플랜:**
 - 프로젝트 5개
 - AI 프리비주얼 월 20회
 - 기본 HDRI/모델 라이브러리
- **프로 플랜 (\$15/월):**
 - 무제한 프로젝트
 - AI 프리비주얼 월 200회
 - 프리미엄 HDRI/모델 접근
 - 고품질 렌더링 (4K)
 - 버전 관리 (30일 히스토리)
- **팀 플랜 (\$50/월, 5명):**
 - 프로 플랜 모든 기능

- 실시간 협업
- 팀 에셋 라이브러리
- 우선 지원
- **엔터프라이즈 (맞춤 견적):**
 - 팀 플랜 모든 기능
 - 온프레미스 옵션
 - SLA 및 전담 지원
 - 커스텀 통합

11.5 구현하지 못한 부분 (향후 개선)

11.5.1 프론트엔드

- **테스트 커버리지:** Vitest + Testing Library 도입 필요
- **성능 측정:** 실제 사용자 데이터 부족 (튜토리얼 완료율, 조명 조정 속도 등)
- **드래그 앤 드롭:** Outliner에서 객체 순서 변경 미구현
- **프리셋 시스템 확장:** 더 많은 조명 프리셋, 마네킹 포즈 프리셋

11.5.2 백엔드

- **WebSocket:** 실시간 협업 미구현
- **버전 관리:** Scene 히스토리 미구현
- **API Rate Limiting:** AI 외 일반 API에도 Rate Limiting 필요
- **로그 시스템:** Winston 등 구조화된 로그 시스템 필요

11.5.3 보안

- **외부 보안 감사:** 전문가의 코드 리뷰 미실시
- **Penetration Testing:** 취약점 스캔 미실시
- **CSRF 보호 강화:** 일부 엔드포인트에만 적용
- **API 키 Rotation:** 정기적인 키 교체 시스템 미구현

11.5.4 UX

- **다국어 지원:** 현재 한국어만 지원
- **다크 모드:** 라이트 모드 옵션 미제공
- **커스터마이징:** 사용자 정의 테마, 단축키 재설정 미지원
- **오프라인 모드:** PWA 캐싱 전략 미구현

12. 프로젝트 성찰 및 배운 점

12.1 기술적 학습

3D 웹 개발

- **Three.js 심화**: PBR 재질, 그림자 매핑, 환경광 이론 학습
- **React-Three-Fiber**: 선언적 3D 프로그래밍의 장점 체득
- **성능 최적화**: 60fps 유지를 위한 렌더링 최적화 기법 습득

상태 관리 패턴

- **Zustand의 우수성**: Redux 대비 보일러플레이트 감소, 러닝 커브 낮음
- **단방향 데이터 흐름**: 복잡한 애플리케이션에서 예측 가능성의 중요성 인식
- **Temporal Middleware**: Undo/Redo 구현의 편리함

백엔드 아키텍처

- **MVCS 패턴**: 관심사의 분리가 유지보수성에 미치는 영향 체감
- **TDD**: 테스트 우선 개발의 중요성 (버그 조기 발견)
- **Mongoose Hooks**: Cascade 삭제, 자동 해싱 등 DRY 원칙 적용

클라우드 인프라

- **Cloudflare R2**: AWS S3 대비 비용 효율성 확인
- **Bull Queue**: 백그라운드 작업의 필요성 및 구현 방법 학습
- **Redis**: 캐싱 및 작업 큐의 핵심 역할 이해

AI 통합

- **Google Gemini API**: 이미지 생성 모델의 활용법 습득
- **프롬프트 엔지니어링**: 원하는 결과를 얻기 위한 프롬프트 작성 기법
- **비동기 처리**: 긴 작업의 사용자 경험 개선 방법

12.2 프로젝트 관리

단계별 계획의 중요성

- **Phase 분리**: 기능을 Phase 단위로 나누어 점진적 개발
- **Milestone 설정**: 명확한 목표로 진행 상황 측정

- 우선순위 지정: P0(긴급) → P3(낮음) 순으로 개발 순서 결정

문서화의 가치

- **API 명세서**: 프론트엔드-백엔드 협업 시 필수
- **아키텍처 문서**: 신규 개발자 온보딩 시간 단축
- **PRD**: 제품 방향성 정렬 및 의사결정 기준

Git 전략

- **Conventional Commits**: 커밋 히스토리 가독성 향상
- **Feature Branch**: main 브랜치 안정성 유지
- 작은 단위 커밋: 자주 커밋하여 변경 추적 용이

12.3 사용자 중심 설계

UX 우선순위

- **튜토리얼의 중요성**: 초보자도 쉽게 시작할 수 있어야 함
- **피드백 시스템**: 모든 액션에 시각적/텍스트 피드백 제공
- **에러 메시지**: 구체적이고 해결 방법을 제시하는 메시지 작성

Cinema 4D 벤치마킹

- **친숙한 UX**: 기존 전문가들이 익숙한 UI 패턴 차용
- **단축키**: 표준 단축키(W/E/R, Ctrl+Z)로 학습 비용 절감
- **Outliner + Properties**: 업계 표준 레이아웃 적용

성능 vs 기능 트레이드오프

- **60fps 유지**: 기능보다 성능 우선 (실시간 렌더링의 핵심)
- **로딩 인디케이터**: 긴 작업은 반드시 진행 상태 표시

12.4 핵심 성과 및 교훈

가장 자랑스러운 성과

1. **Cinema 4D 스타일 Camera-Orbit 동기화**: 복잡한 3D UX 문제를 우아하게 해결
2. **AI 프리비주얼 통합**: 최신 AI 기술을 실용적으로 활용
3. **MVCS 패턴 적용**: 확장 가능한 백엔드 아키텍처 구축

가장 큰 교훈

1. **계획의 중요성**: 사전 계획 없이 코드를 작성하면 기술 부채 누적
2. **사용자 피드백의 가치**: 실제 사용자 테스트 없이는 UX 개선 불가능
3. **점진적 개발**: 작은 단위로 자주 배포하는 것이 대규모 배포보다 안전

다음 프로젝트에 적용할 점

1. 초기부터 **E2E 테스트 도입**: 회귀 버그 방지
2. **성능 모니터링 우선**: 배포 초기부터 성능 데이터 수집
3. **문서화 습관**: 코드 작성과 동시에 문서 업데이트

13. 결론

13.1 프로젝트 요약

LumoStage는 웹 기반 실시간 3D 조명 시뮬레이션 플랫폼으로, 영상 및 영화 제작자들이 촬영 전 조명과 카메라 구도를 미리 시뮬레이션하여 시간과 비용을 절약하고 창의적인 결과물을 만들 수 있도록 돕는 혁신적인 도구입니다.

약 4개월의 개발 기간 동안 **Phase 1부터 Phase 7까지** 총 7단계를 완료하였으며, 다음과 같은 핵심 기능을 구현했습니다:

-  실시간 3D 뷰포트 및 렌더링 (60fps)
-  다양한 조명 타입 및 실시간 제어
-  Cinema 4D 스타일 카메라-Orbit 동기화
-  HDRI 배경 및 3D 오브젝트 관리
-  프로젝트 저장/로드 및 공유 시스템
-  7단계 인터랙티브 튜토리얼
-  전문가용 UI (Outliner, Properties, Toolbar, Undo/Redo)
-  AI 프리비주얼 (Gemini API 통합)

13.2 기술적 하이라이트

- **MERN 스택**: MongoDB, Express, React, Node.js 풀스택 개발
- **Three.js**: WebGL 기반 고품질 3D 렌더링
- **Zustand**: 단방향 데이터 흐름 상태 관리
- **MVCS 패턴**: 관심사의 분리로 유지보수성 향상
- **Cloudflare R2**: S3 호환 스토리지로 비용 80% 절감

- **Bull Queue**: Redis 기반 백그라운드 작업 처리
- **AES-256-GCM**: 사용자 API 키 암호화 보안

13.3 비즈니스 임팩트

LumoStage는 글로벌 3D 시뮬레이션 소프트웨어 시장 (USD 147~168B) 및 조명 설계 소프트웨어 시장 (USD 1.5B → 2.2B) 을 타겟으로 하며, 학생, 독립 제작자, 크리에이터 등 총 약 300만 명 이상의 잠재 사용자를 겨냥합니다.

14. 프로젝트 정보

14.1 저장소 및 실행

실행 방법:

```
# 클라이언트 실행
cd client
npm install
npm run dev
```

```
# 서버 실행
cd server
npm install
npm run dev
```

14.2 기술 문서

- **PRD**: /docs/PRD-Summary.md
- **API 명세**: /docs/api/PROJECT_DASHBOARD_API.md , /docs/api/AI_PREVISUALIZATION_API.md
- **아키텍처**: /docs/architecture/LumoStage-Architecture.md
- **디자인 시스템**: /docs/design/design-strategy.md
- **개발 계획**: /docs/planning/implementation-phases.md